

Summary

Qwen3-8B on one Trainium chip solves a low percentage of Verilog tasks on its first attempt. With our checker in the loop, this rate improves, verified on hidden tests it never saw. Plain retries without feedback perform significantly worse under the same attempt budget.

We built an agent loop where the model writes a module, our checker simulates it, and the failure is translated into targeted evidence for the next attempt. The deciding factor was the feedback, not the model: vague feedback made designs worse, while concrete failing values and an analysis of the logic turned failures into passes.

The problem

A small model writes Verilog that compiles but does the wrong thing. In our initial baseline, 7 of 8 first answers compiled, yet only 3 of 8 passed their tests. These failures were largely logical, not syntactic. The model could also explain a concept correctly and still implement it wrong. Restating the spec did not fix this, which is why the loop's feedback has to show behavior, not repeat definitions.

System

The core checker loop contains a feedback writer and a runner that drives multiple prompt attempts in parallel across team seats. The checker compiles each module with Icarus Verilog, checks its ports against the spec, and simulates it against the visible testbench. The testbench's output provides information on the ratio of errors to total test cases, as well as the location and internal signal values where these errors occurred. On failure, the feedback writer turns the structured result into evidence for the next attempt; only the hidden testbench decides the reported result. The following prompt is constructed by putting together the initial specification, the previous iteration's generated module, and the interpreted feedback.

Findings

The feedback loop improved the performance of Qwen significantly. Without the interpreted feedback from the testbench, Qwen would get stuck going down the wrong path, and would continue to make significant errors even when told where these errors were occurring. Translating these errors into meaningful logic design decisions allowed it to fully flush out modules over multiple iterations.

Future implementations

We are interested in developing a synthesis verification step to the flow that uses yosys to test whether a design is synthesizable in hardware. We have a proof of concept for this idea, but were not able to fully flush it out due to time constraints.